A Machine Learning-Based Toolbox for P4 Programmable Data-Planes

Kaiyi Zhang, *Graduate Student Member, IEEE*, Nancy Samaan, *Member, IEEE*, and Ahmed Karmouch, *Life Member, IEEE*

***Abstract*—Intelligent data-planes (IDPs) can enhance network service performance and adaptation speed by executing one or more machine learning (ML) models directly on the served flows. The real-time ML inference enables line-speed decision-making for some traffic management functionalities. Due to the inherent scarcity of both the computational and memory resources and the strict high-speed per-packet processing demands, existing IDP deployments either realize only a limited set of ML models such as decision trees, or require substantial modifications in the switch hardware. In this paper, we propose INQ-MLT, a novel ML-based management toolbox to address the aforementioned limitations. INQ-MLT delegates the task of training various ML models to the control-plane. The latter adopts a tailored quantization-aware training process to compensate for the effect of precision loss resulting from quantization. The toolbox then employs a quantization mechanism to transform the trained ML model parameters (e.g., weights and activations) from floating-point representations to compact low-precision fixed integer values that can be easily processed and stored in the data-plane. Finally, the trained model is deployed into the IDP pipeline by restricting all its inference operations to basic arithmetic operations. To analyze the performance of INQ-MLT, we quantify the accuracy loss resulting from the quantization step through rigorous theoretical analysis. A proof-of-concept implementation of the proposed toolbox is developed using P4-based software switches. Experiments on two use-cases demonstrate that the deployed quantized models have almost no loss of accuracy when compared to their floating-point counterparts.**

***Index Terms*—Intelligent data-planes, P4, machine learning, neural networks, traffic classification.**

I. Introduction

THE COMPLEXITY of today's network management operations can be largely attributed to the heterogeneity and scale of the managed devices and served traffic. Network management operations include provisioning for strict performance and security guarantees, traffic monitoring and analysis [1], resource configuration [2], and fault management [3]. Machine learning (ML) has demonstrated significant benefits in performing these management tasks [4]. These benefits can be attributed to the ability of ML models to analyze a large set of data at run-time to learn the current characteristics of the hosted traffic or to find optimal network and service configurations.

In current management architectures, these management tasks are mostly performed following the software-defined network (SDN) paradigm [5], [6]. Due to the high computational and storage demands of ML models, their execution has been mostly limited to the control-plane. The latter then must continuously interact with the forwarding devices to collect monitoring data, train the model, and then communicate back the updated operational decisions. The result is a time consuming control-loop that leads to non-negligible delays.

Meanwhile, the introduction of programmable data-plane languages such as P4 [7], [8] has provided an efficient means for the control-plane to customize and adapt the forwarding behavior of packets as carried out by the data-plane. Commonly used P4 architectures such as the Protocol-Independent Switch Architecture (PISA) [9], the v1model [10] and the Portable Switch Architecture (PSA) [11] use a pipeline of match-action tables to modify the forwarding behaviour of flows at a very fine-grained level. Switches running P4 are also equipped with registers that can store per-flow states. Moreover, various protocols, such as P4-runtime [12], [13] have been developed to allow data-plane reconfiguration at runtime.

The increasing popularity of Artificial Intelligence (AI) in various networking management tasks and the gradual adoption of P4 have paved the way for a new vision of intelligent data-planes (IDPs). IDPs enable intelligent traffic analysis at line-speed by deploying ML models on programmable devices [14]. Clearly, offloading ML models to the data-plane will eliminate the needed controller and data-plane communication resulting in a significant decrease in the decision-making latency. IDPs can, hence, effectively improve the management decision quality through real-time processing.

In recent years, several approaches have investigated the feasibility of realizing IDPs with focus on traffic classification [15], DDoS detection [16], [17], heavy flow detection [18], and load balancing [19]. Several efforts have explored offloading of ML tools such as decision trees (DTs) [15], [18], random forests (RFs) [16], [20], binary decision trees (BDTs) [21], K-means [22], support vector machines (SVMs) [22], and binary neural networks (BNNs) [23], [24], [25] to the data plane. In parallel, some other solutions introduced hardware modifications to the switch [26], [27] to host deep-learning models.

Manuscript received 27 October 2023; revised 5 February 2024 and 16 April 2024; accepted 9 May 2024. Date of publication 16 May 2024; date of current version 21 August 2024. This work is funded by a Discovery Grant from the Natural Sciences and Engineering Research Council of Canada (NSERC). The associate editor coordinating the review of this article and approving it for publication was D. Pezaros. *(Corresponding author: Kaiyi Zhang.)*

The authors are with the School of Electrical Engineering and Computer Science, University of Ottawa, Ottawa, ON K1N 6N5, Canada (e-mail: kzhan122@uottawa.ca; nsamaan@uottawa.ca; akarmouc@uottawa.ca).

Digital Object Identifier 10.1109/TNSM.2024.3402074

1932-4537 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: GUILIN UNIVERSITY OF ELECTRONIC TECHNOLOGY. Downloaded on January 10,2025 at 11:20:06 UTC from IEEE Xplore. Restrictions apply.

Unfortunately, implementing ML models that require increased computational capabilities at the data-plane without modifying the switch hardware remains an unaddressed challenge. Current programmable data-planes, such as those using P4, impose several computational restrictions. Particularly, they do not support floating-point operations. This limitation prohibits the direct implementation of neural deep learning based ML models. Even though existing work [28], [29] has been proposed to use lookup-tables for floating-point arithmetic on switches, this can rapidly result in resource exhaustion when implementing large ML models. Similarly, the inability to perform division operations directly using P4 makes it difficult to store flow statistics (e.g., average packet size or its variance) [20].

In this paper, we propose a novel in-network quantized ML toolbox (INQ-MLT).[1] The toolbox is comprised of two components. The first is a quantization-aware ML model training module that executes at the control-plane. The second are quantized ML models that are integrated within the IDP pipeline. INQ-MLT addresses the aforementioned limitations by utilizing quantization techniques and control-data-plane collaboration while achieving efficient decision-making at line-speed. To this end, the main contributions of this article can be summarized as follows.

- We propose a novel toolbox, INQ-MLT, that facilitates the training of ML (e.g., neural and deep learning) models and simplifies the storage and execution of these ML models within the resource-limited IDPs.
- We develop an efficient quantization scheme that allows ML inference to be carried out using integer-only arithmetic on programmable devices. The model is then integrated and executed within the IDP pipeline while meeting its operational and computational constraints.
- To minimize the loss of accuracy resulting from quantization, we adopt a novel training mechanism, quantization-aware training, that forces the training of the model to compensate for any expected loss.
- We provide a rigorous theoretical analysis of how the precision of model parameters impacts the accuracy of model inference and derive the theoretical upper bound on the misclassification rate for the quantized model.

To evaluate the performance of INQ-MLT, we implement two ML models generated by the proposed toolbox on bmv2 software switches [31] using P4. We validate the toolbox through the training and testing of a Multi-Layer Perceptron (MLP) for anomaly detection and a 1D-Convolutional Neural Network (CNN) for multi-label traffic classification use cases respectively.

The remainder of this article is organized as follows; Section II discusses related work. Section III provides an overview of the proposed toolbox while Section IV describes the adopted quantization scheme and explains the quantized ML model execution steps. Section V discusses the P4-based implementation details. Section VI presents the theoretical analysis of the quantization effect on the accuracy. Performance evaluation results are discussed in Section VII. Finally, Section IX concludes the paper and discusses future research directions.

[1]A preliminary work was presented in part at the 2023 IEEE/IFIP Network Operations and Management Symposium (NOMS) [30].

II. Related Work

In this section, we first review research efforts that have contributed to the development of intelligent data planes. We then discuss existing methods for different intelligent traffic management functionalities.

A. Intelligent Data-Planes

Implementing ML models within network devices enables intelligent traffic analysis. The common choice is to implement tree-based models due to their relatively low complexity and intuitive mapping to the match/action tables. The implementation of neural network-based models has also been explored.

1) Tree-Based Implementation: There are three approaches to map tree-based models into P4: *hard-code*, the *level-table* mapping scheme, and the *feature-decision* mapping scheme.

MAP4 [15] provides a framework that hard-codes single decision trees (DTs) using a series of nested *if-else* conditional statements. The *if-else* chain is added to the P4 program. pHeavy [18] is a specialized heavy flow detection scheme based on this approach on the data-plane.

pForest [20] and SwitchTree [32] use *level-table* mapping scheme to deploy DTs on programmable switches, by mapping one match-action stage to one layer of a DT. Specifically, the stages from left to right correspond to each layer of the DT from top to bottom. The table key in each stage is the nodes of the corresponding layer of the DT. The parameter of action is the information of the node on the branch, including feature and threshold. After matching, the comparison between the current feature value and the feature threshold is executed in action, and the next node depends on the comparison result.

IIsy [22], [33] proposes to map DTs using *feature-decision* scheme, which encodes the model using a table per feature. The result (action) is then encoded into a metadata field and indicates a branch taken in the tree. The last stage takes the coded fields of all features from the metadata bus and maps the value to the resulting leaf node. Planter [34] extends this idea, and provides an approach that maps random forest (RF) by overlapping feature encoding over trees within match-action tables. The extended version of Planter [35] also provides an automated framework for mapping ML models on programmable devices. To address the table entry explosion problem under ternary matching, NetBeacon [36] introduces a range marking mechanism to ensure each leaf node only consumes a single ternary entry in the model table.

Mousika [21] modifies the DTs to the binary decision tree (BDT) to save memory usage. The classification rules are bits and thus can be encoded into widely-supported ternary match. It also adopts a teacher-student knowledge distillation architecture to transfer the knowledge from other complicated models to the BDT. By doing so, Mouika not only utilizes their knowledge for better performance but also avoids their complicated deployments on switches. A detailed survey of these approach can be found in [37].

Authorized licensed use limited to: GUILIN UNIVERSITY OF ELECTRONIC TECHNOLOGY. Downloaded on January 10,2025 at 11:20:06 UTC from IEEE Xplore. Restrictions apply.

2) Neural Network-Based Implementation: Implementing Binary Neural Networks (BNNs) on the data-plane has been explored by [23] and [24]. BNN compresses all the weights of a neural network into single bits and only requires simple operations such as XNOR and population count.

Li et al. [25] propose a framework called NNSplit to embed a large BNN in a data plane by splitting the NN layers into several switches. A new protocol called SØREN is designed to carry activation values and support distributed computing between switches.

Zhong et al. [26] propose IOI, a system providing deep neural network (DNN) inference on programmable switches. IOI plugs a novel transceiver module into programmable switches to perform linear operations such as matrix multiplication in the optical domain.

Swamy et al. [27] propose Taurus, a new data-plane architecture for per-packet ML. Taurus aims at implementing DNNs in the data-plane. They extend the PISA with a customized hardware MapReduce block, enabling the execution of DNNs. Swamy et al. [38] also propose HOMUNCULUS, a framework that enables network operators to specify their ML requirements in a declarative way and automatically generates efficient data-plane ML pipelines.

In general, the aforementioned schemes are limited to ML models (e.g., DTs) that can be directly mapped to simple rules within the data-plane match-action tables. Most mapping schemes impose structural limits on the maximum tree depth. For example, the *level-table* mapping scheme used by pForest [20] and SwitchTree [32], uses a match-action stage for each level in the tree. Therefore, the tree depth is restricted by the available number of match-action stage in commercial switch ASICs. Some stages must be allocated for tasks such as calculating flow identifiers, maintaining register indices, updating stateful features, and implementing tree leaves, limiting the maximum tree depth [39]. pForest [20] claims that only relatively shallow trees of depth 4 can be implemented in a Tofino switch. Such tree depth limitation induce performance barriers when dealing with complex inference tasks.

We believe that exploring the applicability of utilizing complex ML models, which have already been utilized by the control-plane, at the data-plane level, can significantly enhance the performance of various network management operations while reducing the control-data plane communication delay.

Finally, orthogonal to our presented solution is IOI [26] and Taurus [27] where the ASIC is modified to support DNN inference. In contrast to these approaches, we aim at offloading ML schemes at the data-plane without modifying the hardware/memory structure of current switches.

B. Intelligent Traffic Management

The complexity of network traffic patterns and the use of encrypted communications are driving the widespread adoption of traffic management based on ML [40]. The ML model customizes itself to the specific network environment by training on the dataset collected for that environment. As the network environment changes, more data is collected, and the model is retrained to reflect these changes, thereby helping to understand relationships between various events that network operators may not be aware of. For example, instead of just matching incoming flows against a statically known set of IP addresses, ML can learn the correlation between fine-grain features (e.g., real-time applications like online meeting often have short inter-arrival time; DDoS attack always involve small packet sizes) to make informed decisions in new and unseen scenarios [38].

1) Traffic Classification: Traffic classification has been applied to a wide range of network operation and management activities from QoS provisioning to performance monitoring as well as resource usage planning [41]. For example, an operator of an enterprise network may want to prioritize traffic for business-critical applications. Traditional port-based [42] and payload-based methods [43], [44] become unreliable with the use of dynamically allocated ports and the ever-increasing security level. Therefore, many ML-based schemes have emerged. Liu et al. [45] propose FS-net, an end-to-end encrypted traffic classification model based on a multi-layer encoder-decoder structure. Shapira and Shavitt [46] propose a CNN-based approach for encrypted traffic classification and application identification by transforming the packet sizes and packet arrival times of each flow into an image, a *FlowPic*. Huoh et al. [47] propose a multi-modal Graph Neural Network (GNN)-based approach for flow-based encrypted traffic classification, which maps traffic flows to graph representations. Wang et al. [48] propose TaTic, a two-phase early classification scheme, where the "easy flows" are quickly classified in the first phase by tree models, and more packets are used to classify the "hard flow" by using Temporal Convolutional Network (TCN). Yun et al. [49] propose a Transport Layer Security (TLS) traffic classification method, NeuTic, by designing a novel neural network equipped with a multi-kernel convolution and a sequence self-attention mechanism.

2) Network Security: Network security consists of protecting the network against cyber-threats that may compromise the network's availability. Doriguzzi-Corin et al. [50] propose Lucid, a CNN-based DDoS attack detection architecture. In Lucid, packet-level attributes were collected within a time-window and then used to generate a spatial data representation that is fed to a CNN. Vinayakumar et al. [51] employ an MLP to develop a flexible and effective intrusion detection system (IDS) for cyberattack detection. Yang and Shami [52] propose a transfer learning and ensemble learning-based IDS for Internet of Vehicles systems using CNNs. Longari et al. [53] propose CANnolo, an IDS based on Long Short-Term Memory (LSTM)-autoencoders to identify anomalies in controller area networks.

It is challenging to meet the throughput and latency requirements of modern networks while performing these ML-based traffic management tasks. In general, these aforementioned schemes deploy their models on control-plane servers. Moving the collected flow statistics from switches to the controller introduces a significant bottleneck for meeting the high throughput requirement. Running ML inference within the network data-plane would avoid data movements and address the challenge.

Authorized licensed use limited to: GUILIN UNIVERSITY OF ELECTRONIC TECHNOLOGY. Downloaded on January 10,2025 at 11:20:06 UTC from IEEE Xplore. Restrictions apply.

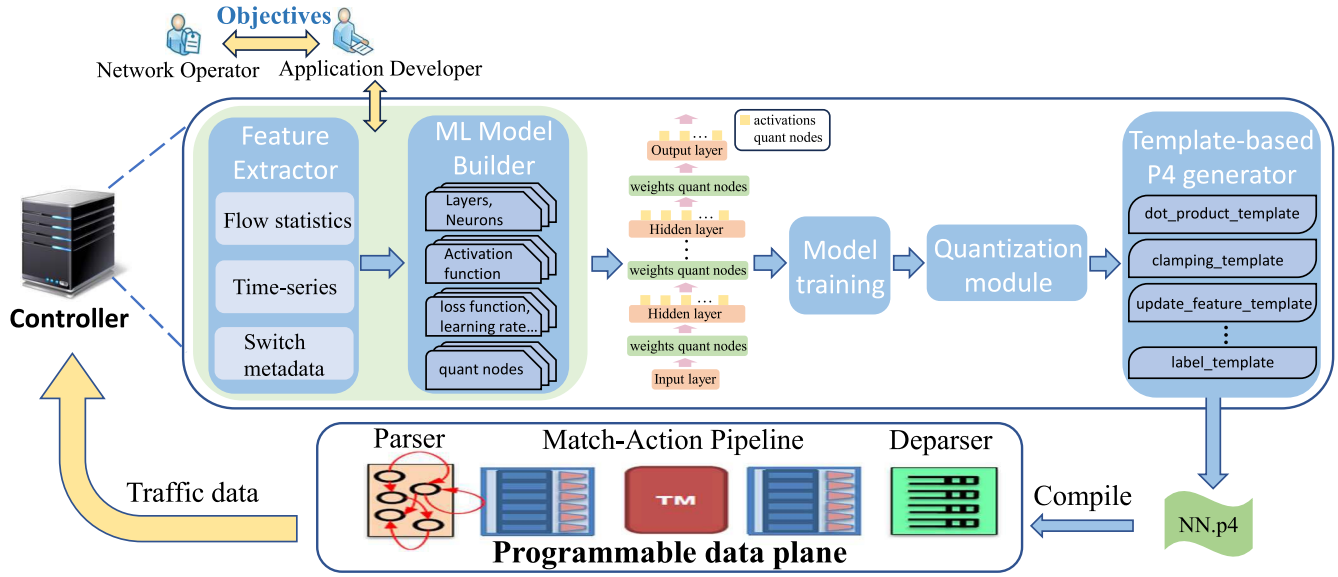


Fig. 1. An overview of INQ-MLT toolbox.

III. Proposed Toolbox Overview

This section provides an overview of the proposed toolbox. As shown in Fig. 1, the controller in the control-plane is responsible for building and training a ML model for network management applications. Network operators describe the desired application behavior to application developers and specify their application objectives (e.g., minimizing false positives in an anomaly detection model or maximizing the throughput of a traffic classification algorithm).

The trained model is then offloaded to the data-plane for various decision-making operations as part of the forwarding pipeline. The forwarding pipeline consists of a parser, multiple match-action tables and a deparser, enabling network devices to customize packet processing within the data-plane. The arriving packet first enters the parser for packet header parsing, then passes through multiple match-action tables for packet manipulation, and finally reaches the deparser for packet serialization. The controller collects the monitored traffic data from the data-plane to periodically retrain the ML model.

The collected traffic data is fed first into the feature extractor to obtain flow features, and application developers are charged with selecting suitable features. Based on the types of features, application developers can manipulate the ML model builder to create appropriate models. Once a floating-point model is obtained and trained, the quantization module generates a quantized ML model that employs P4-supported data types, thereby addressing the limitation of P4 not supporting floating-point numbers. The resulting quantized model utilizes integer weights and bit-oriented arithmetic operations. The mechanism of the quantization module is introduced in Section IV.

The quantized ML model is then fed to the template-based P4 generation module which, in turn, produces the corresponding P4 code by assembling the predefined template of different operations. Finally, the generated P4 code representing the trained model is compiled and mapped to corresponding match-action tables in the IDP. We discuss our proposed packet processing pipeline in Section V. Details pertaining to the remaining components of the toolbox are described in the following sections.

A. Feature Extractor

INQ-MLT identifies a flow $\mathcal{F}$ as a sequence of packets $[\mathcal{F}[i]$ for $i \in [0, 1, \ldots, |\mathcal{F}| - 1]]$ having the same flow ID (e.g., source and destination IP addresses, source and destination ports, and transport-level protocol), where $\mathcal{F}[i]$ denotes the ith packet of the flow. A subflow is denoted as $\mathcal{F}[i{:}j]$ to represent the packet sequence $[\mathcal{F}[i], \mathcal{F}[i+1], \ldots, \mathcal{F}[j-1]]$ and consequently the first n packets of a flow are denoted by $\mathcal{F}[0{:}n]$ or simply $\mathcal{F}[{:}n]$, where $n < |\mathcal{F}|$.

There are three different typical flow features employed in networking traffic management functionalities [54].

1) Standard Flow Statistics: Standard statistical features include the mean, standard deviation, minimum, maximum, of packet length, inter-arrival time, TCP flag count, flow duration, etc. This type of feature is highly abstracted and normally picked by domain experts.

2) Time-Series: Tracking a fixed-size packet-level feature across the initial packets within a flow makes it possible to generate a dynamic-sized time-series feature that characterizes the flow. For example, the sizes of the packets in a flow are valid time-series features. This type of feature includes information about the relationship between two packets.

3) Raw Bytes: The rawest form of flow data, i.e., the actual flow bytes from packet headers and pa

Due to the depth limitation of the parser in parsing from the payload, using P4 to extract a sufficient number of useful bytes from the packet payload presents a challenging task. Additionally, deep examination of payloads using P4 requires clone and recirculation of the packet [55], which may cause a noticeable delay. Therefore, INQ-MLT only considers standard flow statistics and flow time-series features.

Authorized licensed use limited to: GUILIN UNIVERSITY OF ELECTRONIC TECHNOLOGY. Downloaded on January 10,2025 at 11:20:06 UTC from IEEE Xplore. Restrictions apply.

From a practical point of view, classifying a flow after it ends is not a useful task for many types of applications (e.g., intrusion detection). Therefore, to accurately identify an event as soon as possible, INQ-MLT only uses features based on the first n packets $\mathcal{F}[:n]$, where n is a parameter that depends on the application task (e.g., load balance) as well as the number of features. Once the feature type is selected, the next step is to select the relevant features and apply the necessary transformations.

In addition to extracting flow features, the extractor can also retrieve switch metadata (e.g., queue length, hop latency, and ingress timestamp) embedded by In-band Network Telemetry (INT) [56], [57], [58]. As a result, the switch state and the flow's entire history can be obtained to increase the power of the ML model.

B. The ML Model Builder and Model Training

Several factors affect the choice of ML models for network traffic management applications. The most important one is input features. Features directly affect not only the accuracy but input structure/dimension, which influences computational complexity. The choice of input feature and ML model are highly correlated.

Standard flow statistics: The number of statistical features, and consequently the input dimension, is often small. An MLP will be built when standard statistical features are selected.

Time series: A 1D-CNN will be constructed when time-series features are selected because 1D-CNN is capable of capturing temporal correlations between packets.

The application developer can assemble different pre-built neural network basic blocks (combinations of convolution kernels, fully connected layers, activation functions, and more) to design the optimal model for the task at hand.

Then, the generated neural network architecture will be ready for the subsequent training module. The model can be extended to suit the targeted network management operation (e.g., flow classification, flow size prediction, or DDoS prediction). Assume that the neural network consists of an input layer, N hidden layers, and an output layer. INQ-MLT normalizes each input into a zero-mean and unit-variance distribution. The output layer produces the desired outcome (e.g., flow predicted class). For the activation function at the output layer nodes, INQ-MLT utilizes softmax. The neural network uses back-propagation to learn the weights and biases of each layer. Training the model refers to finding values or weights on the links between the nodes of each consecutive layer that can be used in gradually calculating the correct output during model execution.

INQ-MLT inserts additional quantization nodes during the training process. For each link carrying the weight between two nodes, we add a *weights quant node*. Similarly, for each activation function output on each node for all the layers, we add an *activations quant node*. The design of these nodes will be discussed in Section IV-B.

To explain the purpose of these nodes, it is important to note that the deployed ML model at the IDP must satisfy several restrictions. IDPs cannot perform floating point operations and

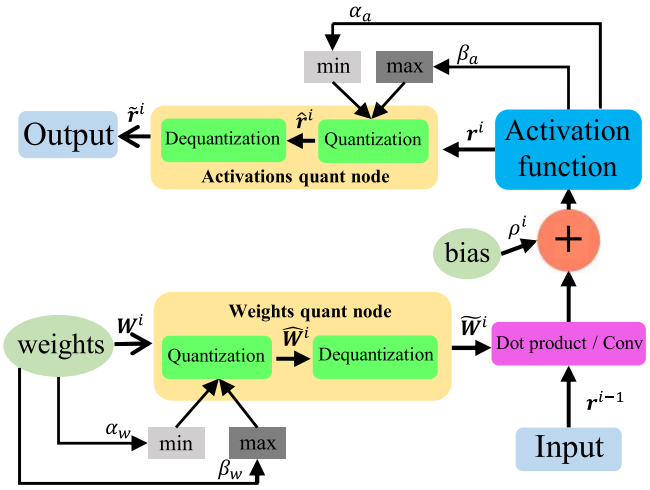


Fig. 2. Added Quant nodes at the controller.

do not support matrix multiplication or looping operations. Hence, the weights of each layer of the ML model stored at the IDP are restricted only to integer representations. On the other hand, it is beneficial to use floating point representations of the features and node weights during training in order to increase the model accuracy. Hence, the training model at the control-plane trains and stores the model using floating-point values. The role of the quant nodes, then, is to train the model to compensate for any precision loss when the floating-point based model at the controller is transformed into a quantized ML model that can be stored at the IDP.

One of our design objectives is to keep the size of the ML model at the IDP small in order to reduce the switch memory consumption. Clearly, there is a trade-off between the model size (e.g., number of hidden layers) and the achieved accuracy, especially in some complex traffic management tasks. Hence, it is up to the developer to configure the desired input/output layers size as well as the number of hidden layers according to the selected network management operation while achieving the desired level of accuracy. Since the model is trained at the control-plane, these configurations can be reached after experimenting with different historical datasets. In our current implementation, we map the entire ML model to a single P4 program that is executed by a single forwarding device. We leave the idea of mapping larger ML models in a distributed manner at several switches as future work.

IV. Quantization Module

The quantization module is employed to convert floating-point numerical model parameters and operations to lower-precision representations. Instead of adopting a 32-bit floating point format to represent weights and activations, the quantized ML model represents weights and activations using more compact formats (e.g., 8-bit integers) and in some cases binary values. Applying quantization to a trained model may introduce a perturbation to the trained model parameters. This can significantly reduce the model accuracy. We employ the quantization-aware training (QAT) technique [59] to address this limitation by inserting quantization nodes, as depicted in Fig. 2. QAT simulates low-precision inference time

Authorized licensed use limited to: GUILIN UNIVERSITY OF ELECTRONIC TECHNOLOGY. Downloaded on January 10,2025 at 11:20:06 UTC from IEEE Xplore. Restrictions apply.

computation in the forward pass of the training process. This section describes how INQ-MLT quantizes the ML model and how the model can be employed.

A. Quantization and Dequantization Steps

At the controller, each quantization step maps a floating-point model parameter r within the continuous range $[R_{min}, R_{max}]$ to a finite integer value $\hat{r}$ between Q_{min} and Q_{max}, respectively. The range set $\{Q_{min}, \ldots, Q_{max}\}$ depends on the bit width B of the resulting integer $\hat{r}$ and the choice of whether it is a signed or an unsigned integer. More precisely,

$$(Q_{min}, Q_{max}) = \begin{cases} (0, 2^B - 1), & \hat{r} \text{ is unsigned} \\ (-2^{B-1}, 2^{B-1} - 1), & \hat{r} \text{ is signed} \end{cases} \quad (1)$$

For each model parameter, four quantization parameters must be defined. The first two are the values $\alpha \geq R_{min}$ and $\beta \leq R_{max}$, such that $[\alpha, \beta]$ denotes the clipping range of r, a bounded range that we are clipping the real values of r with.

Next, we define a real-valued scale S. The scale S specifies the quantization step, or the corresponding real-value distance between two consecutive integers. Using a constant value for S results in a uniform mapping. On the other hand, S can also be represented as a function of the quantized weights resulting in a non-uniform mapping. In essence, a uniform quantization, selects S such that it divides a given range of real values r into an equal number of partitions,

$$S = \frac{\beta - \alpha}{2^B - 1} \quad (2)$$

The fourth parameter is an integer zero-point Z which is the quantization bias. The value of Z controls the mapping symmetry of different r values around its central value. Setting Z to zero results in a symmetric range that is efficient when the values that r takes are not skewed within $[\alpha, \beta]$.

Having defined those quantization constants, we can now proceed with the quantization process, where the quantized integer value $\hat{r}$ is obtained as follows,

$$\hat{r} = \text{clamp}\Big(\Big\lfloor \frac{r}{S} \Big\rceil + Z; Q_{min}, Q_{max}\Big) \quad (3)$$

where r is a real-valued input (activation output or node weight). $\lfloor \cdot \rceil$ is the round-to-nearest integer value operator and clamping is defined as:

$$\text{clamp}(\hat{r}; Q_{min}, Q_{max}) = \begin{cases} Q_{min}, & \hat{r} < Q_{min} \\ \hat{r}, & Q_{min} \leq \hat{r} \leq Q_{max} \\ Q_{max}, & \hat{r} > Q_{max} \end{cases} \quad (4)$$

In other words, any values of $\hat{r}$ that lie outside the range (Q_{min}, Q_{max}) will be clipped to its limits.

Dequantization recovers an approximation $\tilde{r}$ of the real value r using the quantized value $\hat{r}$ using:

$$\tilde{r} = S(\hat{r} - Z) \quad (5)$$

B. ML Quantization-Aware Training

As indicated before, the common approach to obtaining a quantized ML model is to design and train the model using floating point values and operations first. Then, this step is followed by quantizing only its weights. However, this may introduce a perturbation to the trained model parameters and result in a high-level of accuracy loss.

INQ-MLT circumvents this problem by adopting a different approach referred to as quantization-aware training (QAT). QAT recovers the quantization accuracy loss during training by inserting additional quantization (quant) nodes to the ML model during that phase as shown in Fig. 2.

A quantization node is a sequence of quantization (3) and dequantization (5) operations stacked together. By adding those nodes, INQ-MLT introduces the quantization induced errors to the ML training phase. The model is then forced to learn as it is trained how to modify its weights in order to minimize its accuracy loss due to quantization. We note here that these additional nodes are only needed during the training phase and are not part of the ML model deployed to the IDP. Fig. 2 illustrates how quantization nodes are added, respectively, for the quantized weights and activation functions outputs within the ML model. As shown, before the matrix of the weights of every layer is multiplied by the input to the layer, it passes through the *weights quant node* to be quantized and then dequantized. The output of the quant node is a new set of weights with some precision loss. The new weights matrix is multiplied by the input and a bias is added. Similarly, when an activation function is executed, its output is passed to an *activations quant node* before it is passed to the next layers. The mathematical details of the quantization and dequantization operations are explained in Eqn. (3) and Eqn. (5). Hence the quant nodes ensure that the quantization precision loss is fed to the ML model as part of its training.

QAT only simulates the lower precision behavior in the forward pass of the training process, while the back-propagation step remains unchanged.

After the training phase, the quantization process is performed on the pre-trained floating-point model based on the Eqn. (3). It is worth noting here that INQ-MLT treats the quantization ranges $[\alpha, \beta]$ differently for weights and activation function values. More precisely,

- For layer weights, the range is computed statically and is treated as a stored constant when the ML model is deployed to the IDP. It is sufficient for example to simply set $\alpha := \min r, \beta := \max r$.
- For activation functions outputs, (activations for short), the clipping range depends on the flow features, i.e., the input of the neural network. Therefore, the determination of the clipping range of activations quantization often requires a few batches of calibration data. The clipping range can be pre-calculated by running a series of calibration inputs to compute the typical range of activations. INQ-MLT can apply different metrics to find the best range.

C. Deployment of the Quantized ML Model

Once the model is trained, and all its weights are obtained by the control-plane, it is mapped to a corresponding P4 program. The IDP then executes the program using predefined thresholds (e.g., every n packets of a flow or when a certain

Authorized licensed use limited to: GUILIN UNIVERSITY OF ELECTRONIC TECHNOLOGY. Downloaded on January 10,2025 at 11:20:06 UTC from IEEE Xplore. Restrictions apply.

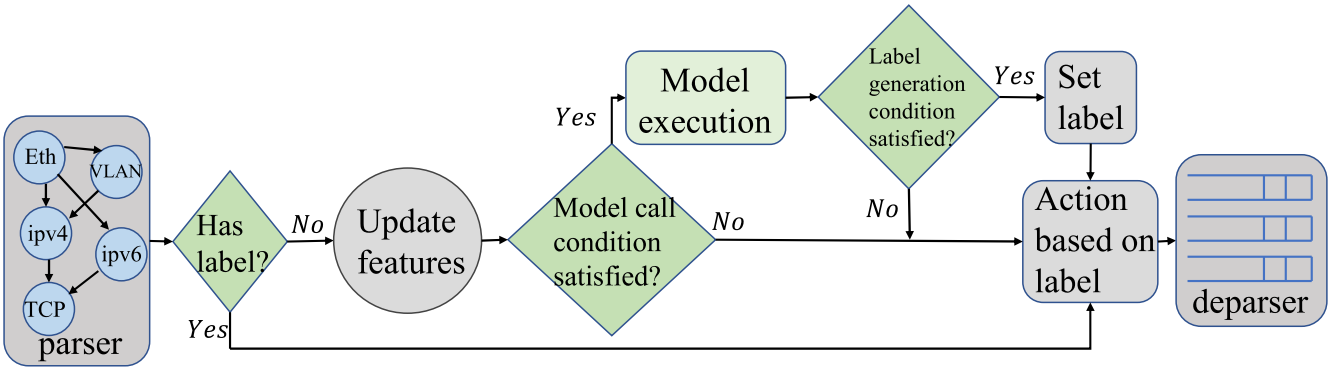


Fig. 3. Packet processing pipeline.

condition is satisfied). The outcome of the ML model then affects the processing of the packet as it passes within the IDP pipeline.

In this section, we demonstrate how the data-plane can store and execute the trained ML model given its constrained computing and memory resources. Assuming that the trained model has *I* Layers with N_i, $i = 1 \cdots I$, nodes at layer *i*. The weights of each layer *i* can be stored in the IDP as an $N_{i-1} \times N_i$ matrix of integer values. These values can be cached in the forwarding device memory or stored in its registers.

The triggering of each layer *i*, involves a multiplication operation between the output vector $\mathbf{r}^{i-1} = (r_1^{i-1}, \ldots, r_{N_{i-1}}^{i-1})$ of size N_{i-1} of the previous layer $i-1$ and the matrix of layer *i* weights, $\mathbf{W}^i = [w_{kl}^i]$ of size $N_{i-1} \times N_i$. Following this step by adding a bias $\boldsymbol{\rho}^i$ produces the vector $\mathbf{r}^i = \mathbf{r}^{i-1} \times \mathbf{W}^i + \boldsymbol{\rho}^i$. Here, $\boldsymbol{\rho}^i$ represents the bias value at layer *i* and $\mathbf{r}^i$ is the output of layer *i* and the ML model output for $i = I$.

Following [60], let $\hat{\mathbf{W}}^i = [\hat{w}_{kl}^i]$ and $\hat{\mathbf{r}}^i = (\hat{r}_1^i, \ldots, \hat{r}_{N_i}^i)$ represent the quantized values of the matrix $\mathbf{W}^i$ and the vector $\mathbf{r}^i$, $i = 1 \cdots I$, respectively, as obtained using Eqn. (3).

From the definition of matrix multiplication, we have:

$$r_k^i = \sum_{l=1}^{N_{i-1}} r_l^{i-1} \times w_{kl}^i + \rho_k^i \tag{6}$$

Substituting for the real-values of the matrix and vectors with their corresponding quantized values, we obtain:

$$S_k^i\left(\hat{r}_k^i - Z_k^i\right) = \sum_{l=1}^{N_{i-1}} S_l^{i-1}\left(\hat{r}_l^{i-1} - Z_l^{i-1}\right) S_{w,kl}^i\left(\hat{w}_{kl}^i - Z_{w,kl}^i\right) + S_b^i \rho_k^i \tag{7}$$

By normalizing the input features and selecting a maximal common clipping range for the elements within each vector and matrix and substituting in (2), the scaling factors and zero-points can be set such that $S_1^i = S_2^i = \cdots = S^i$, $Z_1^i = Z_2^i = \cdots = Z^i$ for every vector $\mathbf{r}^i$ and $S_{w,kl}^i = S_w^i, Z_{w,kl}^i = Z_w^i$, $\forall k, l$. We observe that the bias scale is the product of the scales of the weights and of the input activations, i.e., $S_b^i = S^{i-1} \times S_w^i$. We can then have:

$$\hat{r}_k^i = Z^i + M^i \left(\sum_{l=1}^{N_{i-1}} \left(\hat{r}_l^{i-1} - Z^{i-1}\right)\left(\hat{w}_{kl}^i - Z_w^i\right) + \rho_k^i \right) \tag{8}$$

where the multiplier M^i is defined as

$$M^i := \frac{S^{i-1} \times S_w^i}{S^i} \tag{9}$$

In Eqn. (8), the only non-integer value is that of the multiplier M^i, which can be calculated by the controller before deploying the model to the data-plane. Therefore, during the ML model execution process, we can consider these values as constants expressed and stored using fixed-point representations in the switch.

Clearly, the output of the last layer represents the final results of the ML model, which can be used as part of the data-plane packet pipeline as will be explained in Section V.

V. P4 Generation

In this section, we describe the final step of how our proposed model is integrated within the switch P4 pipeline. The general processing pipeline of an IDP is illustrated in Fig. 3 and is explained in the following sections.

A. Parser

For an arriving packet, it is first mapped into a Packet Header Vector (PHV) by the parser based on the requirement of models. The PHV contains different fields of the packet header, such as the flow ID and associated metadata (e.g., queue length upon packet arrival). Then the PHV is checked if incoming packets contain a label. If a label is present, INQ-MLT executes the corresponding action (e.g., drop the packet). In the case of traffic classification functionalities, the parser also contrasts the packet flow ID against a list of already classified or labeled flows using hash functions. If the packet flow ID is not found in the list it is added. Once hashed, the corresponding stored statistics are retrieved and stored in the associated metadata.

Registers are used to store the aforementioned list of flows, their statistics, and the ML model output for their classification. For example, in the case of an ML model that is used for classifying heavy flows, the first cell in the register will store the flow ID and the last cell will store the label of the flow (e.g., for heavy flow identification, 0 for normal flow and 1 for heavy).

Initially, all cells in the registers are initialized to a default label. When the ML model is triggered to label a flow,

Authorized licensed use limited to: GUILIN UNIVERSITY OF ELECTRONIC TECHNOLOGY. Downloaded on January 10,2025 at 11:20:06 UTC from IEEE Xplore. Restrictions apply.

TABLE I
THE COMMON USED TEMPLATES

Name	Parameters
dot_product_template	The weight matrix $\hat{\mathbf{W}}^i$; the zero-points Z^{i-1}, Z^i and Z_w^i; the multiplier M^i; the bias $\boldsymbol{\rho}^i$.
Conv1D_template	The kernel matrix $\hat{\mathbf{W}}^i$; the zero-points Z^{i-1}, Z^i and Z_w^i; the multiplier M^i; the bias $\boldsymbol{\rho}^i$.
normalize_quantize_template	The mean and variance of the training set.
clamping_template	The bit width applied and the choice of the use of signed or unsigned integer.
update_feature_template	The feature type (count, maximization, minimization or average).
label_template	The number of classes, i.e., how many different categories or classes.

the corresponding cell is modified with the new label. Hash functions in P4 are used to map flows to registers based on their IDs.

B. Update Features

All features and fields that are used by the ML model are also stored in registers. The P4 code that updates some features (e.g., average packet length) is limited to basic integer operations (e.g., addition, subtraction, and hash). To calculate standard statistic features, bit operations are used (e.g., we use right bit shift for divisions). To minimize the number of times these operations are executed, these values are only calculated upon triggering the ML model. For example, to obtain the average packet length, the register maintains the sum of the lengths of all packets as well as the number of packets. Before executing the ML model, the average is calculated once and fed to the model. To increase the accuracy of the calculations, it is preferred to choose the value of the number of packets as a power of two to maintain the accuracy of the division operation as it is carried out using bit-shift operations. For time-series features, rewrite the registers with the features of current packet when the feature of the previous packet is no longer required for model inference.

C. ML Model Execution

After updating features, INQ-MLT checks if the model execution conditions are satisfied. The execution of the ML model can be either partial or complete, depending on the model type. For example, an MLP is fully executed when a specific condition is met (e.g., the *n*th packet arrivals). In contrast, a 1D-CNN can be executed partially upon packet arrival by only executing a single elementwise multiplication. The intermediate results of this multiplication are stored in registers, and labels will only be generated when the label generation conditions are satisfied.

To simplify the mapping from the ML model to P4 code, we create a template library that contains various parameterized templates for commonly used operations. Each template parameters are based on the ML model configuration or are calculated by the control-plane offline (e.g., the weights of each layer, the mean and variance used to normalize the input and the bit width applied). Table I shows some common used templates and their parameters. The P4 developer feeds these

```
1  meta.f1=(((x1-Z^{i-1})*W_11+(x2-Z^{i-1})*W_21+ρ_1^i)*m)>>e+Z^i;
```

Listing 1. P4 Code Fragment of a Dot Product Operation Based on Eqn.(8)

```
1  action i-th_Layer_calculation(){
2      int<32> x1=meta.prevLayerOutput1;
3      int<32> x2=meta.prevLayerOutput2;
4      // ...
5      dot_product_template[C1]();
6      dot_product_template[C2]();
7      /*similar code is omitted*/
8      dot_product_template[Cn]();}
```

Listing 2. P4 Code Fragment of an Action for the Vector-Matrix Multiplication of a Neural Network Layer

```
1  apply {
2      update_feature_template;
3      if(The ML call condition == True){
4          feature1.read(); feature2.read();
5          // Omit codes to extract other features
6          normalize_quantize_template;
7          firstLayer_calculation();
8          clamping_template(meta.1stLayerOutput1);
9          // ...
10         clamping_template(meta.1stLayerOutputn);
11         secondLayer_calculation();
12         meta.class=label_template(meta.output);
13     }}
```

Listing 3. P4 Code Fragment for Mapping a MLP Model With One Hidden Layer

parameters to corresponding templates and assembles these template blocks into larger blocks or full packet pipelines. For example, the template of the operation of Eqn.(8) denoted *dot_product_template* is shown in Listing 1.

Listing 1 shows an example of the *dot_product_template* in layer i with two vectors $[\mathtt{x1}, \mathtt{x2}]$ and $[\hat{w}_{11}^i, \hat{w}_{21}^i]$.

Here the multiplier M^i is expressed using its fixed-point representation $m \times 2^{-e}$, and 2^{-e} is performed by using the right bit shift operation $>>$. $W_{kl} = \hat{w}_{kl}^i - Z_w^i$ can be precomputed since both $\hat{w}_{kl}^i$ and Z_w^i are constants. The calculation result is stored in `meta.f1`.

For the vector-matrix multiplication of an ML layer, we construct an action block which consists of multiple *dot_product_templates* with different parameter combinations, as shown in Listing 2, where `Cn` denotes the parameter combination of the nth column of the weight matrix $\hat{\mathbf{W}}^i$, the nth element of bias vector $\boldsymbol{\rho}^i$, the zero-points Z^{i-1}, Z^i and $\hat{Z}^i$ and the multiplier M^i.

We note that, for the result of each multiplication to be stored and fed to the next layer, they must be clamped using Eqn.(4), which involves conditional statements. Thus, we create *clamping_template*, whose parameter depends on the bit width used and whether the integer is signed or unsigned. Listing 3 shows an example of mapping an MLP model with one hidden layer by assembling the templates in an apply control block.

Before we check whether the ML model call condition is satisfied, we always need to update the flow features by using the *update_feature_template* (line 2 in Listing 3). Its parameter is the feature type, i.e., how do we update the features

Authorized licensed use limited to: GUILIN UNIVERSITY OF ELECTRONIC TECHNOLOGY. Downloaded on January 10,2025 at 11:20:06 UTC from IEEE Xplore. Restrictions apply.

(e.g., count, maximization, minimization or average). Once the neural network is invoked, the values of the input features are fetched from the registers. Before feeding these values to the model, they are normalized. Division operations during normalization are carried out using bit shift operations. The values are then quantized to the integer range set depending on the bit width applied, and used as an input to the first layer in the ML model. *normalize_quantize_template* is used here. Consequently, each layer is triggered by executing the vector-matrix multiplication operation, using one layer at a time.

For the output layer, the classification result is obtained by using *label_template*, whose parameter relies on the number of classes. Recall that softmax is used as the final activation function; therefore, after obtaining the output vector, the label can be determined by identifying the node with the highest value through a straightforward comparison using *if* statements for binary classification. For a multi-class classification problem, it is more scalable to realize the number comparison (i.e., the argmax operation) by designing a ternary matching table, as detailed in [61].

D. Set Label and Apply Action

Once the ML model is executed and the label generation condition is satisfied, its output is used to set the label of the packet flow. P4's match action tables are then used to find the suitable action for the packet belonging to the newly labeled flow. For example, if the packet is labeled as malicious, dropping the packet can be implemented by directly calling the P4 drop primitive or sending it to a deep packet inspection module [44]. Finally, at the end of the pipeline, the deparser assembles the packet data back into a stream of bytes for transmission before their departure.

E. Summary and Discussion

The P4 code is generated by assembling templates for commonly used operations. The P4 mapping of the vector-matrix multiplication consists of many *dot_product_templates*, and each contains multiple basic addition and multiplication operations. Clearly, the computational complexity increases with the growth of the size of the matrix, which increases the ML model inference latency, especially when we increase the model accuracy by using more neurons in each layer. To accelerate the ML model execution, we sketch two potential solutions and leave them as future work. The first solution is to explore the possibility of executing actions in parallel in P4. The second solution is to design an efficient approximate matrix multiplication algorithm [62] which reduces the problem to exact matrix multiplication in a lower-dimensional space.

It is also important to note that, to deploy our toolbox generated P4 programs, the programmable targets must provide direct support for multiplication operations. For those P4 targets that do not provide such support, we believe that with certain extensions, the P4 code generated by our toolbox can be adapted for use. Generally, most P4 targets support the shift and additions. Therefore, the multiplication operation can be decomposed into a number of shifts and additions, as detailed in [63]. Depending on the vector-matrix operations scale, the packets need to be recirculated to the pipeline multiple times to complete all the required operations. Even Taurus [27], which adds an FPGA-based implementation of a custom MapReduce block to programmable switches, enabling the support of vector-matrix multiplication. Our toolbox can adapt to Taurus switches by converting the generated P4 code to the corresponding Spatial language [64] on the FPGA. Finally, replacing vector-matrix multiplication with table lookups can also be explored as a more general approach, as discussed in [65] and [66].

VI. Theoretical Precision Analysis

In this section, we analyze how the precision of the quantized IDP model for activations and weights impact the IDP model inference accuracy theoretically. We employ the DNN precision analysis framework in [67].

A. Notation

Recall that the neural network trained by the controller for the traffic management task has an output layer with size N_I, and softmax is used as the output layer activation function. Therefore, to give the management decision, the neural network would typically have N_I numerical outputs $\{u_j\}_{j=1}^{N_I}$ and the decision would be $y = \arg\max_{j=1,\ldots,N_I} u_j$. Each numerical output is a function of the neural network's weights and activations:

$$u_j = f\big(\{a_h\}_{h\in\mathcal{A}}, \{w_h\}_{h\in\mathcal{W}}\big) \tag{10}$$

for $j = 1, \ldots, N_I$, where $\mathcal{A}$ and $\mathcal{W}$ are the index sets of all activations and weights in the network, respectively. a_h denotes the activation indexed by h and w_h denotes the weight indexed by h.

After applying the quantization module, the activations and weights are quantized to B_A and B_W bits, respectively, and the output u_j is corrupted by quantization noise ε_{u_j} so that:

$$u_j + \varepsilon_{u_j} = f\big(\{a_h + \varepsilon_{a_h}\}_{h\in\mathcal{A}}, \{w_h + \varepsilon_{w_h}\}_{h\in\mathcal{W}}\big) \tag{11}$$

where ε_{a_h} and ε_{w_h} are the quantization noise terms of the activation a_h and weight w_h, respectively. It is standard to assume that $\{\varepsilon_{a_h}\}_{h\in\mathcal{A}}$ are independent uniformly distributed random variables on $[-\frac{\Delta_W}{2}, \frac{\Delta_W}{2}]$, and $\{\varepsilon_{w_h}\}_{h\in\mathcal{W}}$ are independent uniformly distributed random variables on $[-\frac{\Delta_W}{2}, \frac{\Delta_W}{2}]$, with $\Delta_A = 2^{-(B_A-1)}$ and $\Delta_W = 2^{-(B_W-1)}$ [68].

B. Accuracy of Non-quantized Controller Models and Quantized IDP Models

For the non-quantized controller model and its quantized counterpart at the IDP, we define:

1) The Non-Quantized Controller Model Error Probability:

$$p_{e,fl} = \Pr\{Y_{fl} \neq Y\}$$

where Y_{fl} is the output of the non-quantized controller model and Y is the true label.

Authorized licensed use limited to: GUILIN UNIVERSITY OF ELECTRONIC TECHNOLOGY. Downloaded on January 10,2025 at 11:20:06 UTC from IEEE Xplore. Restrictions apply.

2) The Quantized IDP Model Error Probability:

$$p_{e,q} = \Pr\{ Y_q \neq Y \}$$

where Y_q is the output of the quantized IDP model.

3) The Mismatch Probability Between Non-Quantized Controller Models and Quantized IDP Models:

$$p_m = \Pr\{ Y_q \neq Y_{fl} \}$$

Proposition 1: The upper bound for the error probability $p_{e,q}$ of the quantized IDP model is determined by the scenario in which there is no overlap between misclassified samples and samples whose predicted labels are in error due to quantization, as shown below:

$$p_{e,q} \leq p_{e,fl} + p_m \tag{12}$$

Proof: $p_{e,q} = \Pr\{ Y_q \neq Y \}$
$= \Pr\{ Y_q \neq Y, Y_q = Y_{fl} \} + \Pr\{ Y_q \neq Y, Y_q \neq Y_{fl} \}$
$= \Pr\{ Y_{fl} \neq Y, Y_q = Y_{fl} \} + \Pr\{ Y_q \neq Y, Y_q \neq Y_{fl} \}$
$\leq p_{e,fl} + p_m.$

From the above proposition, it can be seen that it requires estimating the mismatch probability to obtain an upper bound error probability of the quantized IDP model.

C. Quantization Analysis

In quantization noise analysis, it is standard to ignore the cross-products of quantization noise terms as their contribution is negligible. Therefore, using Taylor's theorem, the total quantization noise at the output of the quantized IDP model can be expressed as [67], [69]:

$$\varepsilon_{u_i} = \sum_{h \in \mathcal{A}} \varepsilon_{a_h} \frac{\partial u_i}{\partial a_h} + \sum_{h \in \mathcal{W}} \varepsilon_{w_h} \frac{\partial u_i}{\partial w_h} \tag{13}$$

By evaluating the probability of any pair of outputs $(u_i < u_j)$ that flipped due to quantization errors $\Pr(u_i + \varepsilon_{u_i} > u_j + \varepsilon_{u_j})$, we can obtain an analytical upper bound on the mismatch probability p_m between the non-quantized controller and quantized IDP models.

Proposition 2: Given B_A and B_W, the mismatch probability p_m between the non-quantized controller model and its quantized counterpart at the IDP is upper bounded as follows:

$$p_m \leq \Delta_A^2 E_A + \Delta_W^2 E_W \tag{14}$$

where

$$E_A = \mathbb{E}\left[\sum_{\substack{j=1 \\ j \neq Y_{fl}}}^{N_I} \frac{\sum_{h \in \mathcal{A}} \left| \frac{\partial \left(u_j - u_{Y_{fl}} \right)}{\partial a_h} \right|^2}{24 \left| u_j - u_{Y_{fl}} \right|^2} \right] \tag{15}$$

and

$$E_W = \mathbb{E}\left[\sum_{\substack{j=1 \\ j \neq Y_{fl}}}^{N_I} \frac{\sum_{h \in \mathcal{W}} \left| \frac{\partial \left(u_j - u_{Y_{fl}} \right)}{\partial w_h} \right|^2}{24 \left| u_j - u_{Y_{fl}} \right|^2} \right] \tag{16}$$

E_A and E_W are the activation and weight quantization noises, respectively. The expectations $\mathbb{E}$ are taken over a random input and $\{a_h\}_{h \in \mathcal{A}}$, $\{u_j\}_{j=1}^{N_I}$, and Y_{fl} are thus random variables.

The detailed proof of this proposition can be found in [67].

The first term in (14) characterizes the impact of quantizing activations on the overall IDP model accuracy while the second characterizes that of weight quantization. We observe that the mismatch probability p_m increases as Δ_A^2 and Δ_W^2 becomes larger. This can be attributed to the fact that smaller precision results in more mismatch. Proposition 2 provides additional insight: the mismatch probability decreases exponentially with precision, because $\Delta_A = 2^{-(B_A - 1)}$ and $\Delta_W = 2^{-(B_W - 1)}$.

To obtain the quantities in the expectations in (15) and (16), it is necessary to perform one forward pass on an estimation set. During this forward pass, the numerical outputs are recorded, followed by one backward pass to probe all relevant derivatives. Consequently, (14) can be readily calculated. Then, we simply combine (12) with (14) to obtain an estimate of the accuracy of the quantized IDP model.

VII. Performance Evaluation

We have implemented the performance of INQ-MLT on a network with bmv2 software switches [31]. We selected anomaly detection [70] and traffic classification [71] as the use-case scenarios for validating the performance of our proposed toolbox.

A. Dataset and Feature Extraction

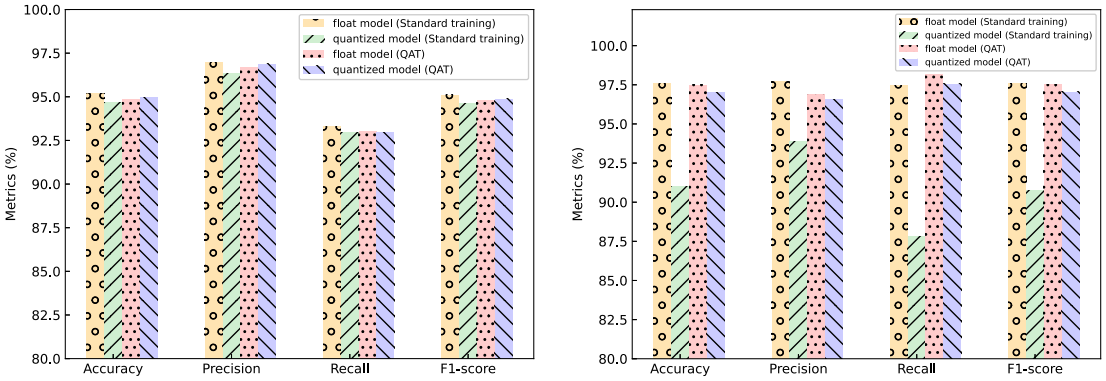
Anomaly detection: The *CICIDS2017* [70] dataset is provided by the Canadian Institute for Cybersecurity, University of New Brunswick Canada. This dataset consists of data collected from several days of network activity. It contains both benign and malicious (e.g., flows representing DDoS attacks) flows. We select Tuesday, Wednesday, and Friday traces. We extract the flow features of the first eight packets to collect the needed input features vector for the ML model, which is then executed as part of the eighth packet's processing pipeline. The used dataset is highly unbalanced. Following [50], we utilize an under-sampling technique that randomly removes records from the dominant class to ensure unbiased learning and classification. We build MLPs to perform anomaly detection. MLP represents the basis for implementing artificial and deep neural networks and has been widely used as a benchmark in various network management tasks [72], [73]. Several stateful features are used to train the MLP, as listed in Table II.

Traffic classification: The *ISCX VPN-nonVPN* [71] dataset is utilized for the task of classifying traffic according to the function types. It contains six function types of network traces (email, chat, file transfer, P2P, VoIP, and streaming). We build a 1D-CNN to perform this traffic classification task. Three time-series features, i.e., packet length, inter-arrival time, and TCP window size, of the first eight packets, are used to train the 1D-CNN.

These datasets are stored in PCAP file format which facilitates the use of dpkt[2] to extract the features.

[2]https://github.com/kbandla/dpkt

Authorized licensed use limited to: GUILIN UNIVERSITY OF ELECTRONIC TECHNOLOGY. Downloaded on January 10,2025 at 11:20:06 UTC from IEEE Xplore. Restrictions apply.



(a) MLP-1 (a hidden layer with 16 nodes)

(b) MLP-2 (2 hidden layers with 16 and 8 nodes respectively)

Fig. 4. Classification performance for MLP.

TABLE II
FEATURES USED FOR MODEL TRAINING

Model	Feature	Description
MLP	PSH	Cumulative number of occurrences of the TCP flag PSH
	RST	Cumulative number of occurrences of the TCP flag RST
	SYN	Cumulative number of occurrences of the TCP flag SYN
	FIN	Cumulative number of occurrences of the TCP flag FIN
	Max_Pkt_Len	Size of the largest packet
	Mean_Pkt_Len	Average size of packets
1D-CNN	Pkt_Len	The time-series of packet length
	IAT	The time-series of inter-arrival time between two adjacent packets
	TCP win_size	The time-series of TCP window size

B. Experiment Setup

For each use-case, the dataset is split into a training (60%), a validation (20%), and a test (20%) sets. The model training, validation, and quantization operations are performed by the control-plane using TensorFlow Lite.[3] We created a simple network topology consisting of two end-hosts connected by one forwarding device in the Mininet[4] environment. The Linux network traffic tool, Tcpreplay,[5] was used to replay the real network traces. *iperf*[6] is used to measure the throughput. The bit width B is set as 8.

MLP configuration: Two MLPs are developed: MLP-1 and MLP-2. Specifically, MLP-1 is composed of 1 hidden layer with 16 hidden units; whereas MLP-2 is composed of 2 hidden layers with 16 and 8 hidden units, respectively. Each hidden layer has a ReLU activation function.

CNN configuration: The 1D-CNN consists of 1 hidden layer with 16 kernels of size 2×3 and stride of 1, a flatten layer, and a fully-connected layer with 6 nodes.

C. Evaluation Metrics

For a specific traffic class i under analysis, classified test samples are divided into three categories:

- *True Positives for class i* (TP_i) - the number of flows that are correctly classified for $class_i$.
- *False Positives for class i* (FP_i) - the number of flows that are classified as belonging to $class_i$ but are not generated by $class_i$.
- *False Negatives for class i* (FN_i) - the number of flows that are classified as not belonging to $class_i$ but are indeed generated by $class_i$.

The performance metrics used for evaluating our model are accuracy, recall, precision, and F1 score.

1) Recall: It is defined as the rate of the flows correctly classified as $class_i$ in the total $class_i$ flows:

$$Recall_i = \frac{TP_i}{TP_i + FN_i}$$

2) Precision: It is defined as the rate of the real flows belonging to $class_i$ in all the flows classified as $class_i$:

$$Precision_i = \frac{TP_i}{TP_i + FP_i}$$

3) F1-Score: The F1-score is the harmonic mean of precision and recall, and it measures the balance between precision and recall and is defined as:

$$F1 - score_i = 2 \times \frac{Recall_i \times Precision_i}{Recall_i + Precision_i}$$

4) Accuracy: The overall accuracy metrics are used to assess the overall classification performance of the classifier, which is defined as the ratio of the total number of samples that are correctly classified by the model to the total number of samples:

$$Accuracy = \frac{\sum_{i \in classes} TP_i}{\sum_{i \in classes} (TP_i + FP_i)}$$

D. Classification Result

In our comparative analysis, we evaluate the proposed Quantization-Aware Training (QAT) scheme alongside the standard training scheme, where quantization nodes are not added during the training process.

For MLP-1, as depicted in Fig. 4(a), it was observed that both the QAT scheme and the standard training scheme exhibited nearly identical levels of performance, with no significant

[3] https://www.tensorflow.org/lite
[4] http://mininet.org/
[5] https://tcpreplay.appneta.com
[6] https://iperf.fr

Authorized licensed use limited to: GUILIN UNIVERSITY OF ELECTRONIC TECHNOLOGY. Downloaded on January 10,2025 at 11:20:06 UTC from IEEE Xplore. Restrictions apply.

TABLE III
CLASSIFICATION PERFORMANCE FOR 1D-CNN

	Overall Accuracy	Metric	Email	Chat	Streaming	Voip	File	P2P	Weighted Average
float model (standard training)	0.7230	Precision	0.7536	0.4583	0.7458	0.6816	0.6933	0.9176	0.7142
		Recall	0.8125	0.1964	0.4490	0.8085	0.8739	0.9176	0.7230
		F1-score	0.7820	0.2750	0.5605	0.7397	0.7732	0.9176	0.7040
quantized model (standard training)	0.6623	Precision	0.6974	0.4000	0.6429	0.5730	0.7407	0.9390	0.6651
		Recall	0.8281	0.1429	0.4592	0.8564	0.5042	0.9059	0.6623
		F1-score	0.7571	0.2105	0.5357	0.6866	0.6000	0.9222	0.6420
float model (QAT)	0.7230	Precision	0.8387	0.3721	0.6667	0.6912	0.7405	0.9176	0.7146
		Recall	0.8125	0.2857	0.4898	0.7979	0.8151	0.9176	0.7230
		F1-score	0.8254	0.3232	0.5647	0.7407	0.7760	0.9176	0.7145
quantized model (QAT)	0.7246	Precision	0.9123	0.3953	0.6712	0.6741	0.7422	0.9176	0.7203
		Recall	0.8125	0.3036	0.5000	0.8032	0.7983	0.9176	0.7246
		F1-score	0.8595	0.3434	0.5731	0.7330	0.7692	0.9176	0.7176

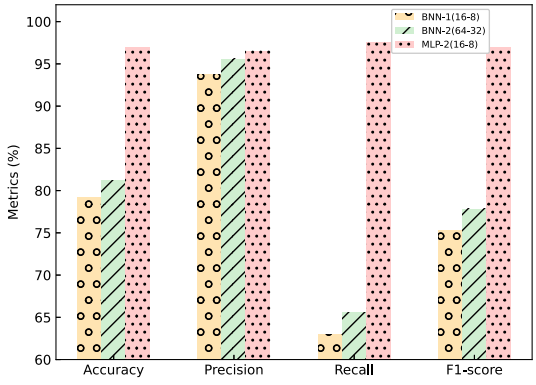


Fig. 5. Comparison between BNN and 8-bit precision MLP.

difference in accuracy. In contrast, Fig. 4(b) shows when assessing the performance of QAT for MLP-2, characterized by two hidden layers, notable distinctions emerged. The quantized model associated with the QAT scheme demonstrated a notably higher accuracy in comparison to the standard training scheme, wherein the latter exhibited a reduction in accuracy. This shows that QAT is more effective in improving the accuracy of deeper neural network architectures, such as MLP-2, compared to shallower ones like MLP-1. This is because the quantization error accumulates gradually with the increase of hidden layers, leading to accuracy loss.

Table III presents similar results for the multi-label traffic classification tasks based on 1D-CNN and it shows the performance results on metrics for all 6 traffic function types. For the QAT scheme-associated models, compared to the non-quantized float model, we note that the quantized model has almost no loss in performance metrics. This validates the effectiveness of QAT.

We also conduct an evaluation to compare the classification performance of our quantized neural network with the binary neural network (BNN). As outline in [39], even simplified versions of BNNs are onerous to deploy into commodity programmable switches: a very basic BNN with two layers of 64 and 32 neurons already exhausts the resources of an Intel Tofino ASIC, while yielding poor inference results. Therefore, we train two BNNs for the anomaly detection task: BNN-1 with the identical architecture as our MLP-2 and BNN-2 conforming to the structure with 64 and 32 neurons referenced in [74]. The comparative analysis depicted in Fig. 5 illustrates the performance difference between the two BNNs and the 8-bit precision MLP-2. As expected, the neural network with 8-bit precision exhibits higher accuracy compared to the two BNNs. BNN-2 employs a higher number of neurons, however, the enhancement in its classification performance is marginal. The distinctions between the BNN and the 8-bit precision neural network performance can be attributed to the following reasons. Specifically, an 8-bit precision allows for a wider range of weight values compared to binary weights, enhancing the model's expressiveness. Furthermore, binary weights (1-bit) inherently introduce quantization errors during training and inference, potentially leading to a loss of information. In contrast, the 8-bit precision offers a higher level of granularity, reducing quantization errors and preserving more information during computations.

E. INQ-MLT in Switches

The previous experiments are performed with Python. We now further evaluate our scheme on bmv2 software switches. We implement MLP-1 on the software switch. We compare the performance of our proposed architecture against that of a centralized controller-based ML model at runtime. In this model, the data-plane sends collected features to the controller, which executes the ML model, and then communicates back the classification decision to the data-plane. Fig. 6 provides a comparison between the controller-based ML model and our P4 implementation of the ML model. As depicted, there is almost no accuracy difference.

Furthermore, we evaluate the classification performance of our approach in comparison to DTs with level-table mapping scheme adopted by [20], [32]. As claimed by pForest [20], only shallow trees of depth 4 can be implemented due to resource limitation. Therefore, we performed a comparison to determine the depth required for a DT to reach equivalent accuracy as a neural network with only one hidden layer. We can see from Fig. 6, to achieve the same accuracy as MLP-1, for the DT with level-table mapping scheme, a depth of 6 is needed.

In terms of the throughput of the IDP pipeline, we first note that the ML inference delay in the controller consists of three parts: the transmission delay of sending features from the

Authorized licensed use limited to: GUILIN UNIVERSITY OF ELECTRONIC TECHNOLOGY. Downloaded on January 10,2025 at 11:20:06 UTC from IEEE Xplore. Restrictions apply.

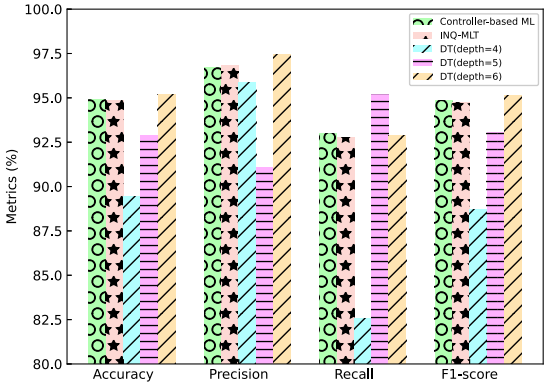


Fig. 6. The performance of INQ-MLT, controller-based ML and DT for anomaly detection.

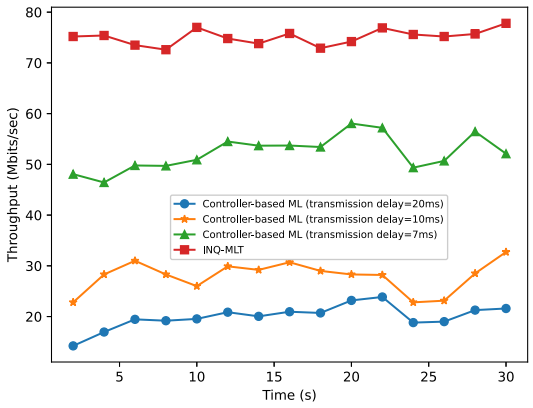
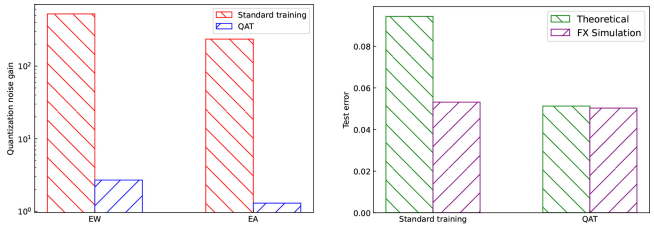


Fig. 7. Throughput comparison.

forwarding device to the controller, the ML execution delay in the controller and finally the transmission delay of returning the result back to the switch. Fig. 7 plots the throughput of a scenario where every packet triggers the execution of the ML model for both the IDP of our INQ-MLT and that of the controller against various values of the one-way transmission delay between the controller and the switch. The choice of transmission delay is based on [75]. We neglect the ML execution delay at the controller due to the controller's high computational capability. As depicted, the controller-based ML exhibits an increase in throughput as the transmission delay decreases, aligning with expectations. In contrast, our proposed approach, operating without causing any transmission, consistently achieves higher throughput compared to other three controller-based schemes. Since bmv2 was not developed as a production-grade switch, a better performance should be expected whilst running on a hardware switch. The plot demonstrates that INQ-MLT achieves a significant reduction in the traffic management decision-making process, especially when the transmission delay between the controller and the switch is high.

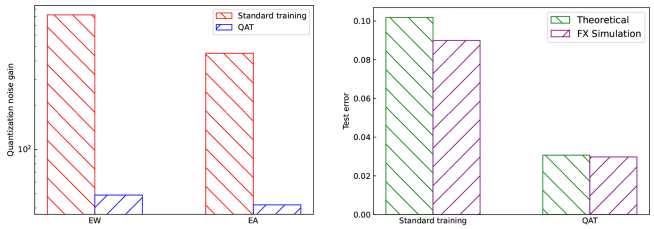
F. Precision Analysis Result

Figs. 8, 9, 10 show the quantization noises computed by (15) and (16) and the theoretical and simulation error rates, respectively. Both quantization noises E_A and E_W for



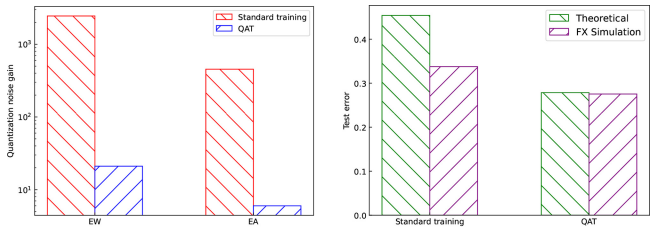
(a) Quantization noise (b) Error rate

Fig. 8. Comparison of quantization noises and error rates between standard training and QAT Schemes for MLP-1.



(a) Quantization noise (b) Error rate

Fig. 9. Comparison of quantization noises and error rates between standard training and QAT Schemes for MLP-2.



(a) Quantization noise (b) Error rate

Fig. 10. Comparison of quantization noises and error rates between standard training and QAT Schemes for 1D-CNN.

standard training scheme are higher than the corresponding values for the QAT scheme, which means the impact of quantizing activations and weights on the overall accuracy is smaller for the QAT scheme compared to the standard training scheme. The theoretical bound successfully upper bound the error rate obtained through fixed-point simulations (FX simulation) for both schemes, which validates the analysis conducted in Section VI. The theoretical error rate is smaller for the QAT scheme compared to the standard training scheme, indicating that applying the QAT scheme offers a higher theoretical guarantee on the model's accuracy.

VIII. Discussion

In this section, we discuss some important considerations regarding the use of the toolbox.

Updating ML models during runtime: After an ML model is deployed, it is essential to update it periodically to adapt to changes in traffic patterns and maintain model performance. There are two possible updates scenarios: updating model weights and model modification (e.g., changing the number of selected flow features). Weights update can be realized by storing weights in reconfigurable memory (e.g., registers) and

Authorized licensed use limited to: GUILIN UNIVERSITY OF ELECTRONIC TECHNOLOGY. Downloaded on January 10,2025 at 11:20:06 UTC from IEEE Xplore. Restrictions apply.

updating them atomically, which is analogous to updating values and without affecting the forwarding pipeline. Modifying the ML model by changing the flow features is expected to be an infrequent event but it remains to be a significant challenge to perform this modification at runtime. Existing solutions such as FlexCore [76] and IPSA [77] aims to achieve such P4 runtime programmability but they all require modifications to the existing switch architecture.

ML performance: Normally, a trade-off exists between model size, inference accuracy, and resource consumption. Increasing the number of features may require additional hidden layer neurons to capture the characteristics of these features, leading to a larger model and a higher computing resources. In some cases, while a large model can achieve higher accuracy, it may impact the line-rate execution of the P4-based target. Therefore, in certain tasks, it is acceptable to tolerate some accuracy loss to maintain the switches' processing speed at line-rate. Additionally, pruning [78] can be employed to minimize the computing resources needed for executing multiplication operations by removing parameter that do not impart the inference accuracy.

Feature Storage: Registers are the most widely used externs and are supported by most P4 targets. They are typically implemented in Static Random-Access Memory (SRAM) in certain P4 hardware [79]. Registers are used to maintain per-flow states and compute flow-level features. P4 uses hash functions to allocate the memory cells for each flow. Depending on the characteristics of distinct features, the width of memory cells employed for storage may vary; for instance, allocating 4 bits for the TCP flag and 16 bits for the packet size is sufficient. For the features used to train our MLP in the anomaly detection task, each flow requires 48 bits to store the necessary features. Thus, 1MB of memory can accommodate approximately 170k active flows. Unfortunately, the limited size of registers and the large number of flows may result in frequent hash collisions. By tracking the arrival time of the last packet and employing a timeout mechanism, it becomes possible to remove inactive flows information from the registers, hence significantly reducing collisions. Additionally, the model weights are also stored in registers, and the memory required to store the weights of each layer depends on both the weight matrix and the applied bit width.

IX. Conclusion and Future Work

In this paper, we presented INQ-MLT, an innovative toolbox designed to facilitate the training and deployment of machine learning models within intelligent data-planes. In INQ-MLT, the controller adopts a quantization-aware training mechanism to construct and train an ML model. The proposed mechanism compensates for the accuracy loss due to quantization by enforcing several quantization and de-quantization steps during the training process. Once trained, the weights and activations of the trained model are quantized and mapped from their floating point representations to more compact integer representations. A P4 program integrates the quantized model inference is compiled and deployed to the IDP. Simulation results demonstrated the feasibility of deploying the quantized models and showed that they can effectively classify ongoing flows at runtime after the first few packets. In future work, we will examine the feasibility of deploying larger deep learning models in a distributed manner at the data plane.

References

[1] M. Abbasi, A. Shahraki, and A. Taherkordi, "Deep learning for network traffic monitoring and analysis (NTMA): A survey," *Comput. Commun.*, vol. 170, pp. 19–41, Mar. 2021.

[2] P. Poullie, T. Bocek, and B. Stiller, "A survey of the state-of-the-art in fair multi-resource allocations for data centers," *IEEE Trans. Netw. Service Manag.*, vol. 15, no. 1, pp. 169–183, Mar. 2017.

[3] Y. Yu et al., "Fault management in software-defined networking: A survey," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 1, pp. 349–392, 1st Quart., 2019.

[4] R. Boutaba et al., "A comprehensive survey on machine learning for networking: Evolution, applications and research opportunities," *J. Internet Services Appl.*, vol. 9, no. 1, pp. 1–99, 2018.

[5] D. Kreutz, F. M. V. Ramos, P. E. Verissimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proc. IEEE*, vol. 103, no. 1, pp. 14–76, Jan. 2015.

[6] Y.-C. Wang and S.-Y. You, "An efficient route management framework for load balance and overhead reduction in SDN-based data center networks," *IEEE Trans. Netw. Service Manag.*, vol. 15, no. 4, pp. 1422–1434, Dec. 2018.

[7] P. Bosshart et al, "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 3, pp. 87–95, 2014.

[8] A. Liatifis, P. Sarigiannidis, V. Argyriou, and T. Lagkas, "Advancing SDN from openflow to P4: A survey," *ACM Comput. Surveys*, vol. 55, no. 9, pp. 1–37, 2023.

[9] N. McKeown, "Pisa: Protocol independent switch architecture," in *P4 Workshop*, pp. 1–22, 2015.

[10] "v1model." Accessed: Jan. 14, 2024. [Online]. Available: https://github.com/p4lang/p4c/blob/main/p4include/v1model.p4

[11] (P4.org Architect. Working Group, Santa Clara, CA, USA). *P416 Portable Switch Architecture (PSA)*. Accessed: Jan. 14, 2024. [Online]. Available: https://p4.org/p4-spec/docs/PSA-v1.2.html

[12] (P.a Work. Group, Stanford, CA, USA). *Initial Draft Specification for P4 Runtime, P4 Language Consortium*. Oct. 2017. [Online]. Available: https://p4.org/p4-runtime/

[13] T. Osiński, H. Tarasiuk, P. Chaignon, and M. Kossakowski, "A runtime-enabled p4 extension to the open vSwitch packet processing pipeline," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 3, pp. 2832–2845, Sep. 2021.

[14] R. Parizotto, B. L. Coelho, D. C. Nunes, I. Haque, and A. Schaeffer-Filho, "Offloading machine learning to programmable data planes: A systematic survey," *ACM Comput. Surveys*, vol. 36, no. 1, pp. 1–34, 2023.

[15] B. M. Xavier, R. S. Guimarães, G. Comarela, and M. Martinello, "MAP4: A pragmatic framework for in-network machine learning traffic classification," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 4176–4188, Dec. 2022.

[16] B. Coelho and A. Schaeffer-Filho, "Backorders: Using random forests to detect DDoS attacks in programmable data planes," in *Proc. 5th Int. Workshop P4 Eur.*, 2022, pp. 1–7.

[17] K. Zhang, N. Samaan, and A. Karmouch, "A two-stage confidence-based intrusion detection system in programmable data-planes," in *Proc. IEEE Global Commun. Conf. GLOBECOM*, 2023, pp. 1–6.

[18] X. Zhang, L. Cui, F. P. Tso, and W. Jia, "*pHeavy*: Predicting heavy flows in the programmable data plane," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 4, pp. 4353–4364, Dec. 2021.

[19] C. Zheng, B. Rienecker, and N. Zilberman, "QCMP: Load Balancing via In-network Reinforcement Learning," in *Proc. 2nd ACM SIGCOMM Workshop Future Internet Rout. Address.*, 2023, pp. 35–40.

[20] C. Busse-Grawitz, R. Meier, A. Dietmüller, T. Bühler, and L. Vanbever, "pForest: In-network inference with random forests," 2019, *arXiv:1909.05680*.

[21] G. Xie et al., "Empowering in-network classification in programmable switches by binary decision tree and knowledge distillation," *IEEE/ACM Trans. Netw.*, vol. 32, no. 1, pp. 382–395, Feb. 2024.

[22] Z. Xiong and N. Zilberman, "Do switches dream of machine learning? Toward in-network classification," in *Proc. 18th ACM Workshop Hot Topics Netw.*, 2019, pp. 25–33.

Authorized licensed use limited to: GUILIN UNIVERSITY OF ELECTRONIC TECHNOLOGY. Downloaded on January 10,2025 at 11:20:06 UTC from IEEE Xplore. Restrictions apply.

[23] G. Siracusano et al., "Re-architecting traffic analysis with neural network interface cards," in *Proc. 19th USENIX Symp. Netw. Syst. Design Implement. (NSDI 22)*, 2022, pp. 513–533.

[24] Q. Qin, K. Poularakis, K. K. Leung, and L. Tassiulas, "Line-speed and scalable intrusion detection at the network edge via federated learning," in *Proc. IFIP Netw. Conf. (Networking)*, 2020, pp. 352–360.

[25] D. C. Li, M. R. Maulana, and L.-D. Chou, "NNSplit-SøREN: Supporting the model implementation of large neural networks in a programmable data plane," *Comput. Netw.*, vol. 222, Feb. 2023, Art. no. 109537.

[26] Z. Zhong et al., "IOI: In-network optical inference," in *Proc. ACM SIGCOMM Workshop Opt. Syst.*, 2021, pp. 18–22.

[27] T. Swamy, A. Rucker, M. Shahbaz, I. Gaur, and K. Olukotun, "Taurus: A data plane architecture for per-packet ML," in *Proc. 27th ACM Int. Conf. Architect. Support Program. Lang. Oper. Syst.*, 2022, pp. 1099–1114.

[28] M. Jose, K. Lazri, J. François, and O. Festor, "Stateful InREC: Stateful in-network real number computation with recursive functions," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 1, pp. 830–845, Mar. 2023.

[29] P. Cui et al., "Enabling in-network floating-point arithmetic for efficient computation offloading," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 4918–4934, Dec. 2022.

[30] K. Zhang, N. Samaan, and A. Karmouch, "An intelligent data-plane with a quantized ML model for traffic management," in *Proc. IEEE/IFIP Netw. Oper. Manag. Symp. (NOMS)*, 2023, pp. 1–9.

[31] "Behavioral model version 2." Accessed: May 22, 2022. [Online]. Available: https://github.com/p4lang/behavioral-model

[32] J.-H. Lee and K. Singh, "SwitchTree: In-network computing and traffic analyses with random forests," *Neural Comput. Appl.*, 2020, to be Published.

[33] C. Zheng et al., "IIsy: Hybrid in-network classification using programmable switches," *IEEE/ACM Trans. Netw.*, early access, Feb. 16, 2024, doi: 10.1109/TNET.2024.3364757.

[34] C. Zheng and N. Zilberman, "Planter: Seeding trees within switches," in *Proc. Poster Demo Sessions (SIGCOMM '21)*, New York, NY, USA, 2021, pp. 12–14.

[35] C. Zheng et al., "Planter: Rapid prototyping of in-network machine learning inference," *ACM SIGCOMM Comput. Commun. Rev.*, 2024, to be Published.

[36] G. Zhou, Z. Liu, C. Fu, Q. Li, and K. Xu, "An efficient design of intelligent network data plane," in *Proc. 32nd USENIX Security Symp. (USENIX Security 23)*, 2023, pp. 1–18.

[37] C. Zheng, X. Hong, D. Ding, S. Vargaftik, Y. Ben-Itzhak, and N. Zilberman, "In-network machine learning using programmable network devices: A survey," *IEEE Commun. Surveys Tuts.*, early access, Dec. 19, 2023, doi: 10.1109/COMST.2023.3344351.

[38] T. Swamy, A. Zulfiqar, L. Nardi, M. Shahbaz, and K. Olukotun, "Homunculus: Auto-generating efficient data-plane ML pipelines for datacenter networks," in *Proc. 28th ACM Int. Conf. Architect. Support Program. Lang. Oper. Syst.*, 2023, pp. 329–342.

[39] A. T.-J. Akem, M. Gucciardo, and M. Fiore, "Flowrest: Practical flow-level inference in programmable switches with random forests," in *Proc. INFOCOM IEEE Conf. Comput. Commun.*, 2023, pp. 1–10.

[40] E. Papadogiannaki and S. Ioannidis, "A survey on encrypted network traffic analysis applications, techniques, and countermeasures," *ACM Comput. Surveys*, vol. 54, no. 6, pp. 1–35, 2021.

[41] S. Rezaei and X. Liu, "Deep learning for encrypted traffic classification: An overview," *IEEE Commun. Mag.*, vol. 57, no. 5, pp. 76–81, May 2019.

[42] T. T. T. Nguyen and G. Armitage, "A survey of techniques for Internet traffic classification using machine learning," *IEEE Commun. Surveys Tuts.*, vol. 10, no. 4, pp. 56–76, 4th Quart., 2008.

[43] C. Xu, S. Chen, J. Su, S.-M. Yiu, and L. C. K. Hui, "A survey on regular expression matching for deep packet inspection: Applications, algorithms, and hardware platforms," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 4, pp. 2991–3029, 4th Quart., 2016.

[44] J. Hypolite, J. Sonchack, S. Hershkop, N. Dautenhahn, A. DeHon, and J. M. Smith, "Deepmatch: Practical deep packet inspection in the data plane using network processors," in *Proc. 16th Int. Conf. Emerg. Netw. Exp. Technol.*, 2020, pp. 336–350.

[45] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, 2019, pp. 1171–1179.

[46] T. Shapira and Y. Shavitt, "FlowPic: A generic representation for encrypted traffic classification and applications identification," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 1218–1232, Jun. 2021.

[47] T.-L. Huoh, Y. Luo, P. Li, and T. Zhang, "Flow-based encrypted network traffic classification with graph neural networks," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 2, pp. 1224–1237, Jun. 2023.

[48] Y. Wang, H. He, Y. Lai, and A. X. Liu, "A two-phase approach to fast and accurate classification of encrypted traffic," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1071–1086, Jun. 2023.

[49] X. Yun, Y. Wang, Y. Zhang, C. Zhao, and Z. Zhao, "Encrypted TLS traffic classification on cloud platforms," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 164–177, Feb. 2023.

[50] R. Doriguzzi-Corin, S. Millar, S. Scott-Hayward, J. M.-d. Rincon, and D. Siracusa, "Lucid: A practical, lightweight deep learning solution for DDoS attack detection," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 2, pp. 876–889, Jun. 2020.

[51] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. 7, pp. 41525–41550, 2019.

[52] L. Yang and A. Shami, "A transfer learning and optimized CNN based intrusion detection system for Internet of Vehicles," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2022, pp. 2774–2779.

[53] S. Longari, D. H. Nova Valcarcel, M. Zago, M. Carminati, and S. Zanero, "CANnolo: An anomaly detection system based on LSTM autoencoders for controller area network," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 1913–1924, Jun. 2021.

[54] I. Akbari et al., "A look behind the curtain: Traffic classification in an increasingly encrypted Web," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 5, no. 1, pp. 1–26, 2021.

[55] S. Gupta, D. Gosain, M. Kwon, and H. B. Acharya, "Deep4r: Deep packet inspection in P4 using packet recirculation," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, 2023, pp. 1–10.

[56] L. Tan et al., "In-band network telemetry: A survey," *Comput. Netw.*, vol. 186, Feb. 2021, Art. no. 107763.

[57] R. B. Basat, S. Ramanathan, Y. Li, G. Antichi, M. Yu, and M. Mitzenmacher, "Pint: Probabilistic in-band network telemetry," in *Proc. Annu. Conf. ACM Special Interest Group Data Commun. Appl., Technol., Architec., Protocols Comput. Commun.*, 2020, pp. 662–680.

[58] S. Tang, D. Li, B. Niu, J. Peng, and Z. Zhu, "Sel-INT: A runtime-programmable selective in-band network telemetry system," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 2, pp. 708–721, Jun. 2020.

[59] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," 2021, *arXiv:2103.13630*.

[60] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 2704–2713.

[61] J. Yan et al., "Brain-on-switch: Towards advanced intelligent network data plane via NN-driven traffic analysis at line-speed," 2024, *arXiv:2403.11090*.

[62] M. Ghashami, E. Liberty, J. M. Phillips, and D. P. Woodruff, "Frequent directions: Simple and deterministic matrix sketching," *SIAM J. Comput.*, vol. 45, no. 5, pp. 1762–1792, 2016.

[63] K. Razavi, G. Karlos, V. Nigade, M. Mühlhäuser, and L. Wang, "Distributed DNN serving in the network data plane," in *Proc. 5th Int. Workshop P4 Eur.*, 2022, pp. 67–70.

[64] D. Koeplinger et al, "Spatial: A language and compiler for application accelerators," in *Proc. 39th ACM SIGPLAN Conf. Program. Lang. Design Implement.*, 2018, pp. 296–311.

[65] D. Blalock and J. Guttag, "Multiplying matrices without multiplying," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 992–1004.

[66] Y. Jeon, B. Park, S. J. Kwon, B. Kim, J. Yun, and D. Lee, "BiQGEMM: Matrix multiplication with lookup table for binary-coding-based quantized DNNs," in *Proc. Int. Conf. High Perform. Comput., Netw., Stor. Anal. (SC20)*, 2020, pp. 1–14.

[67] C. Sakr, Y. Kim, and N. Shanbhag, "Analytical guarantees on numerical precision of deep neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 3007–3016.

[68] C. Caraiscos and B. Liu, "A roundoff error analysis of the LMS adaptive algorithm," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 32, no. 1, pp. 34–41, Feb. 1984.

[69] J. L. Holt and J.-N. Hwang, "Finite precision error analysis of neural network hardware implementations," *IEEE Trans. Comput.*, vol. 42, no. 3, pp. 281–290, Mar. 1993.

[70] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. ICISSp*, 2018, pp. 108–116.

[71] G. Draper-Gil, A. H. Lashkari, M. S. Islam Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related," in *Proc. 2nd Int. Conf. Inf. Syst. Security Privacy (ICISSP)*, 2016, pp. 407–414.

Authorized licensed use limited to: GUILIN UNIVERSITY OF ELECTRONIC TECHNOLOGY. Downloaded on January 10,2025 at 11:20:06 UTC from IEEE Xplore. Restrictions apply.

[72] O. Aouedi, K. Piamrat, and B. Parrein, "Ensemble-based deep learning model for network traffic classification," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 4124–4135, Dec. 2022.
[73] M. D. Mauro, G. Galatro, and A. Liotta, "Experimental review of neural-based approaches for network intrusion management," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 4, pp. 2480–2495, Dec. 2020.
[74] G. Siracusano and R. Bifulco, "In-network neural networks," 2018, *arXiv:1801.05731*.
[75] A. Alowa and T. Fevens, "Towards minimum inter-controller delay time in software defined networking," *Procedia Comput. Sci.*, vol. 175, pp. 395–402, Aug. 2020.
[76] J. Xing, K.-F. Hsu, M. Kadosh, A. Lo, Y. Piasetzky, A. Krishnamurthy, and A. Chen, "Runtime programmable switches," in *Proc. 19th USENIX Symp. Netw. Syst. Design Implement. (NSDI 22)*, 2022, pp. 651–665.
[77] Y. Feng et al., "Enabling in-situ programmability in network data plane: From architecture to language," in *Proc. 19th USENIX Symp. Netw. Syst. Design Implement. (NSDI 22)*, 2022, pp. 635–649.
[78] T. Liang, J. Glossner, L. Wang, S. Shi, and X. Zhang, "Pruning and quantization for deep neural network acceleration: A survey," *Neurocomputing*, vol. 461, pp. 370–403, Oct. 2021.
[79] F. Hauser et al., "A survey on data plane programming with P4: Fundamentals, advances, and applied research," *J. Netw. Comput. Appl.*, vol. 212, Mar. 2023, Art. no. 103561.

Kaiyi Zhang (Graduate Student Member, IEEE) received the B.Eng. degree from the China University of Petroleum, Qingdao, China, in 2018, and the M.Sc. degree from University of Ottawa, Ottawa, ON, Canada, in 2020, where he is currently pursuing the Ph.D. degree in electrical and computer engineering. His research interests include software defined networks, programmable data-planes, and machine learning.

Nancy Samaan (Member, IEEE) received the B.Sc. and M.Sc. degrees from the Department of Computer Science, Alexandria University, Egypt, and the Ph.D. degree in computer science from the University of Ottawa, Canada, in 2007, where she is currently a Professor with the School of Electrical Engineering and Computer Science. Her current research interests include software defined networks, programmable data-planes, information centric networks, network resource management, and autonomic communications.

Ahmed Karmouch (Life Member, IEEE) received the M.S. and Ph.D. degrees in computer science from the University of Paul Sabatier, Toulouse, France, in 1976 and 1979, respectively. He is a Professor with the School of Electrical Engineering and Computer Science, University of Ottawa. He also held an Industrial Research Chair with the Ottawa Carleton Research Institute and Natural Sciences and Engineering Research Council. He has been the Director of the Ottawa Carleton Institute for Electrical and Computer Engineering. He is involved in several projects with industry and government laboratories in Canada and Europe. His current research interests are in, named data networks, software defined networks, cloud computing, in-network computing, and programmable data-planes. He organized several conferences and workshops, and edited several books. He served as a Guest Editor for *IEEE Communications Magazine* and *Computer Communications*.

Authorized licensed use limited to: GUILIN UNIVERSITY OF ELECTRONIC TECHNOLOGY. Downloaded on January 10,2025 at 11:20:06 UTC from IEEE Xplore. Restrictions apply.